

i Velkommen

Eksamen i INF122 – Tidlig eksamen våren 2024

Velkommen til eksamen i INF122. Denne eksamenen teller 70% av sluttkarakteren.

Antall oppgaver: 7, hvorav 2 flervalgsoppgaver og 5 programmeringsoppgaver

Det blir ikke trukket poeng for uriktige svar på flervalgsoppgavene. Så det lønner seg alltid å svare.

Programmeringsoppgavene er delt opp i deloppgaver. Alle deloppgaver skal svares på i samme kodeboks, nederst på oppgavesiden. **Marker hver deloppgave tydelig.**

Noen deloppgaver ber deg om en beregning eller forklaring. Disse skriver du også i kodeboksen.

Bruk tiden godt! Dersom du sitter fast på en oppgave kan du fortsette med de andre oppgavene og komme tilbake til den oppgaven senere.

Dersom svaret ikke er helt riktig, kan du fortsatt få poeng for en delvis løsning.

Tillatte hjelpemidler på eksamen: ingen.

Lykke til!

1 Verdier og typer

a) Hva er verdien av uttrykket: `snd (9,3)` ?

b) Hva er verdien av uttrykket: `tail [3,5,7]` ?

c) Hva er verdien til uttrykket `length $ "Hello" ++ "World"`?

Velg ett alternativ

- ☐ 10
- ☐ "HelloWorld"
- ☐ 5
- ☐ "HWeolrlod"
- ☐ [('H','W'),('e','o'),('l','r'),('l','l'),('o','d')]

d) Hva er typen til uttrykket: `tail "123"`?

Velg ett alternativ

- ☐ Integer
- ☐ Char
- ☐ [Integer]
- ☐ String

e) Hva er typen til funksjonen `head` fra standardbiblioteket?

Velg ett alternativ

- ☐ `[a] -> a`
- ☐ `a -> [a]`
- ☐ `a -> a`
- ☐ `[a] -> [a]`

f) Hva er verdien av uttrykket `map reverse (words "en to tre")`?

Velg ett alternativ

- ☐ "ne ot ert"
- ☐ ["tre","to","en"]
- ☐ "tre to en"
- ☐ ["ne","ot","ert"]

g) Hvilken type er en gyldig typing av uttrykket `zipWith (+) [1..]` ?

Velg ett alternativ

- ☐ [Integer]
- ☐ [Integer] -> Integer
- ☐ [Integer] -> [Integer]
- ☐ Integer -> Integer
- ☐ Integer

Maks poeng: 14

2 Typer, kinds og monader

a) Hva er en mulig typing av uttrykket `Right (2,["remain","silent"])`?

Velg ett alternativ

- ☐ Either a (Integer,[String])
- ☐ Either Integer [String]
- ☐ Maybe (Integer,String)
- ☐ Right (String, [Integer])

b) Hva er en mulig typing av uttrykket `\xs -> [(x,x) | x <- xs]` ?

Velg ett alternativ

- ☐ `[x] -> [(x,x)]`
- ☐ `[(a,a)] -> [a]`
- ☐ `x -> [(x,x)]`
- ☐ `[(a,a) -> (a,a)]`

c) Hva er en mulig typing av uttrykket `zipWith ($)`?

Husk at `zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]` og at `($) :: (a -> b) -> a -> b`.

Velg ett alternativ

- ☐ `(a -> b) -> [[a]] -> [[b]]`
- ☐ `[a -> b] -> [a -> b]`
- ☐ `([a] -> [b]) -> [a -> b]`
- ☐ `[a -> b] -> [a] -> [b]`

d) Hvilken kind har `Maybe Integer`?

Velg ett alternativ

- ☐ *
- ☐ Functor
- ☐ $* \rightarrow *$
- ☐ Maybe

e) Hvilken kind har Either (Maybe Integer)?

Velg ett alternativ

- ☐ Monad
- ☐ $* \rightarrow *$
- ☐ Either
- ☐ $* \rightarrow * \rightarrow *$

f) Hva er verdien til uttrykket ("hei" >>= (take 3 . repeat)) ?

Husk at $(>>=) :: m a \rightarrow (a \rightarrow m b) \rightarrow m b$ er den monadiske bind operasjonen og at lister er en monade. $repeat :: a \rightarrow [a]$ er funksjonen som lager en uendelig liste med kopier av et element.

g) Hva er typen til $(>>= putStrLn)$?

Husk at $(>>=) :: m a \rightarrow (a \rightarrow m b) \rightarrow m b$ er den monadiske bind operasjonen

Velg ett alternativ

- ☐ $IO () \rightarrow String$
- ☐ $IO String \rightarrow IO ()$
- ☐ $String \rightarrow IO ()$
- ☐ $IO String$

Maks poeng: 7

3 Grunnleggende IO

Skriv et Haskell program som leser inn en setning og skriver ut hvor mange ord den består av. Du vil ha bruk for IO funksjoner slik som `putStr` og `getLine`, samt `words :: String -> [String]`.

Eksempelkjøring:

Skriv en setning: Eksamen gikk bedre enn forventet.
Setningen bestod av 5 ord.

Skriv ditt svar her

1	
---	--

Maks poeng: 6

4 Lister

I denne oppgaven skal du vise at du behersker listemanipulasjon i Haskell. Du kan fritt bruke funksjoner fra standardbiblioteket. For eksempel kan følgende funksjoner komme til nytte:

```
fst :: (a,b) -> a
snd :: (a,b) -> b
map :: (a -> b) -> [a] -> [b]
all :: (a -> Bool) -> [a] -> Bool
any :: (a -> Bool) -> [a] -> Bool
filter :: (a -> Bool) -> [a] -> [a]
zip :: [a] -> [b] -> [(a,b)]
```

a) **Skriv en funksjon evens :: [Integer] -> [Integer] som gir alle partallene i en liste.** For eksempel `evens [1,5,2,6,7,8] = [2,6,8]`. Husk at funksjonen `even :: Integer -> Bool` sjekker om et heltall er et partall.

b) **Skriv en funksjon allEqual :: (Eq a) => [a] -> Bool som sjekker om alle elementer i en liste er like.** For eksempel `allEqual [1,1,1] = True` og `allEqual [1,0,1] = False`. Returner True dersom listen er tom.

c) **Skriv en funksjon unique :: (Eq a) => [a] -> Bool som sjekker om alle elementer i en liste er ulike.** For eksempel `unique [1,2,3] = True` og `unique [1,0,1] = False`. Returner True dersom listen er tom.

d) Funksjonen `zip :: [a] -> [b] -> [(a,b)]` setter sammen to lister parvis til en liste med par. Man kan tenke seg en funksjon som går andre veien, og tar en liste av par og lager to lister: `unzip :: [(a,b)] -> ([a],[b])`. For eksempel blir `unzip [(1,2),(3,4),(5,6)] = ([1,3,5],[2,4,6])`. **Skriv funksjonen unzip.**

e) La `unzip` være slik som i forrige oppgave. **Er det sant at `zip (fst (unzip ps)) (snd (unzip ps)) = ps` for en hver liste med par `ps :: [(a,b)]` ?** Husk å begrunne svaret.

f) La `unzip` være slik som i d). **Er det sant at `unzip (zip as bs) = (as,bs)` for alle lister `as :: [a]` og `bs :: [b]` ?** Husk å begrunne svaret.

Skriv ditt svar her

1	
---	--

--	--

Maks poeng: 13

5 Tic-tac-toe funksjoner

```
import Data.Map (Map)
import qualified Data.Map as Map
Map.insert :: Ord k => k -> a -> Map k a -> Map k a
Map.lookup :: Ord k => k -> Map k a -> Maybe a
Map.fromList :: Ord k => [(k,a)] -> Map k a
```

I denne oppgaven skal vi implementere funksjoner relatert til spillet “Tre-på-rad”. Spillet spilles på et 3×3 rutenett og de to spillerne tures om å sette merker i rutene, X (første spiller) eller O (andre spiller), for å forsøke å få tre på rad. På sin tur kan man kun sette sitt eget merke i en rute på brettet som ikke allerede har et merke.

```
data Mark = X | O
  deriving (Eq, Show)
type Position = (Integer, Integer)
type Board = Map Position Mark
```

Vi bruker typen Position = (Integer,Integer) for å representere posisjoner på brettet. Vi begrenser oss til posisjoner med koordinater fra 0 til 2, slik at vi får et 3×3 spillebrett. Data typen Mark (se nedenfor) representerer ett merke, og Board = Map Position Mark representerer tilstanden til spillebrettet som en mapping fra posisjoner til merker. Husk at et Map ikke behøver ha verdier for hver nøkkel, så det er mulig å representere også tomme ruter.

For eksempel vil brettet som ser slik ut:

	0	1	2
0	X	X	O
1		O	
2		O	X

representeres av:

```
exampleBoard :: Board
exampleBoard = Map.fromList [ ((0, 0), X)
                             , ((0, 1), X)
                             , ((0, 2), O)
```

```
, ((1, 1), O)
, ((2, 1), O)
, ((2, 2), X) ]
```

a) **Bruk listekomprehensjon til å lage listen allPositions :: [Position] med alle posisjoner på brettet.**

b) **Skriv en funksjon isValidMove :: Board -> Position -> Bool som sjekker om det er mulig å sette et merke på den gitte posisjonen på brettet i følge reglene.**

c) **Skriv en funksjon move :: Board -> Position -> Mark -> Maybe Board som gir en “kanskje verdi” av det nye brettet etter et merke er satt.** Nothing signaliserer at det ikke var mulig å sette merke på den gitte posisjonen.

Vi skal nå se på hvordan vi kan beregne om en spiller har vunnet. Dette kan gjøres på flere måter. Her skal vi ta utgangspunkt i å lage en oversikt over alle "streaks", altså alle lister med tre ruter som gir tre på rad. Deretter sjekker vi om alle rutene i en streak har samme merke.

Her er listen over alle mulige streaks:

```
streaks :: [[Position]]
streaks = [ [(0,0), (0,1), (0,2)],
             [(1,0), (1,1), (1,2)],
             [(2,0), (2,1), (2,2)], -- Rows
             [(0,0), (1,0), (2,0)],
             [(0,1), (1,1), (2,1)],
             [(0,2), (1,2), (2,2)], -- Columns
             [(0,0), (1,1), (2,2)],
             [(0,2), (1,1), (2,0)] ] -- Diagonals
```

d) **Skriv en funksjon hasStreak :: [Positions] -> Board -> Maybe Mark som sjekker om alle posisjonene i en liste har samme merke.** Dersom noen av de listede rutene er tomme, eller ikke alle har samme merke på brettet, så returneres Nothing. Dersom alle de listede rutene inneholder X så returnes Just X, og dersom alle inneholder O, så returneres Just O.

e) **Skriv en funksjon won :: Board -> Maybe Mark, som avgjør om et gitt brett er vunnet av en spiller.** Funksjonen gir Nothing hvis ingen spillere har funnet, Just X dersom første spiller har vunnet og Just O dersom den andre spilleren har vunnet. Gjør dette ved å iterere over alle mulige streaks og bruk hasStreak funksjonen for å sjekke om de er en vinnende streak.

Skriv ditt svar her

1	
---	--

--	--

Maks poeng: 10

6 Tic-tac-toe parsing

Vi fortsetter å jobbe med Tre-på-rad. Nå skal du skrive en parser for brettet.

Du skal parse strenger som representerer tilstanden til brettet. Hver linje representerer en rad i brettet. X representer første spiller, O (stor o) representerer andre spiller og understrek ("_") representerer en tom rute.

Husk at '\n' :: Char representerer linjeskift. For eksempel representerer "_X_\nO_X\nOX_" brettet:

```
_X_
O_X
OX_
```

Parsing

I dette kurset har vi brukt en enkel parser type:

```
newtype Parser a
  = Parser {runParser :: String -> Maybe (String, a)}
```

I disse oppgavene kan du anta at instanser for `Functor`, `Applicative`, `Alternative` og `Monad` klassene er gitt for `Parser`. Du vil også kanskje ha bruk for følgende operatorer fra disse:

```
(<$>) :: (a -> b) -> Parser a -> Parser b
(<*>) :: Parser (a -> b) -> Parser a -> Parser b
(*>) :: Parser a -> Parser b -> Parser b
(<*) :: Parser a -> Parser b -> Parser a
(<|>) :: Parser a -> Parser a -> Parser a
empty :: Parser a
pure/return :: a -> Parser a
(Typene er instansiert for Parser)
```

For å parse en sekvens kan det være greit å bruke `sequence :: [Parser a] -> Parser [a]`.

Du kan også ta for gitt en funksjon `char :: Char -> Parser Char` som lager en parser som kun parser det gitte tegnet. For eksempel `char 'c'` parser kun 'c' og ingenting annet.

a) **Skriv en `pMark :: Parser Mark` som parser ett enkelt merke.**

b) **Skriv en `pCell :: Parser (Maybe Mark)` som parser en celle på brettet.** Denne gir `Just X` for "X", `Just O` for "O" eller `Nothing` for "_".

c) **Skriv `pBoard :: Parser Board` som parser et helt 3×3 brett.** *Hint: Det kan være enklere å lage brettet som en liste og så bruke `Map.fromList :: (Ord a) => [(a,b)] -> Map a b` for å lage mappingen.*

Skriv ditt svar her

--	--

Maks poeng: 10

7 Filsystem som induktiv datatype

I denne oppgaven du vise at du forstår induktive datastrukturer og kan skrive funksjoner ved hjelp av rekursjon på slike.

Vi definerer en induktiv datastruktur for å representere et filsystem, med filer og mapper (directories):

```
data FileSystem = File String Integer -- File name and size in bytes
                | Directory String [FileSystem] -- Directory name and contents
                deriving (Show, Eq)
```

-- A path to specific file is represented as a list of Strings:
type Path = [String]

```
exampleFileSystem :: FileSystem
exampleFileSystem = Directory "root"
  [ File "notes.txt" 120
  , File "todo.txt" 80
  , Directory "projects"
    [ File "projectA.txt" 200
    ]
  ]
```

a) **Skriv en funksjon `totalSize :: FileSystem -> Integer` regner ut den totale størrelsen av alle filer i filsystemet i antall bytes.** Hint: Bruk patternmatching og rekursjon.

b) **Skriv en funksjon `findFile :: String -> FileSystem -> Path` som leter igjennom filsystemet etter en fil med et gitt navn og returnerer stien frem til den.** For eksempel blir `findFile "projectA.txt" exampleFileSystem = ["root","projects","projectA.txt"]`. En tom liste signaliserer at filen ikke ble funnet i filsystemet.

Skriv ditt svar her

--	--

Maks poeng: 10